

MACHINE LEARNING · CLASSIFICATION

K-Nearest Neighbors

Classifying **Iris flowers** by asking their closest neighbours to **vote**.

BUILT WITH



scikit-learn · StandardScaler · KNN

How KNN carves up the space



What is K-Nearest Neighbors?

KNN is one of the simplest ML classifiers. It doesn't learn a formula — it **memorises the examples**, then labels a new point by asking its **k closest neighbours to vote**.



Majority vote

The most common label among the k nearest points becomes the prediction.



Lazy & instance-based

No training step really happens — it just stores data and decides at predict time.



Distance-driven

“Closest” means smallest distance, so every feature must be on a fair scale.



The whole idea

$$k = 5$$

*“Look at the 5 nearest known flowers,
then go with the majority.”*

How a prediction is made

1

Measure distance

From the new flower to every flower the model already knows.

2

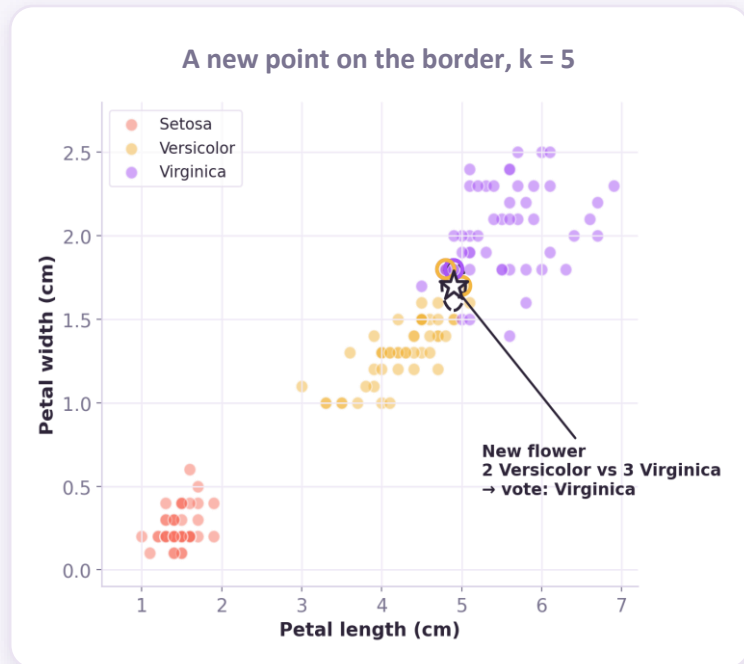
Keep the k closest

Sort by distance and take the k nearest (here, $k = 5$).

3

Take a majority vote

The most common label among those neighbours wins.



Choosing k and measuring distance



k = how many neighbours vote

A small k reacts to every nearby point (noisy); a large k is smoother but can blur the borders between classes. An odd k avoids tie votes.



Distance = straight-line (Euclidean)

KNN ranks known points by Euclidean (straight-line) distance. The closest points are treated as the most similar, so they get a say in the vote.

IN THIS CODE

5

neighbours vote (k = 5)

Euclidean distance

odd k → no tie votes

Why we scale the features

Because KNN compares **distances**, a feature with bigger numbers (like sepal length) would unfairly dominate one measured in small numbers (like petal width).

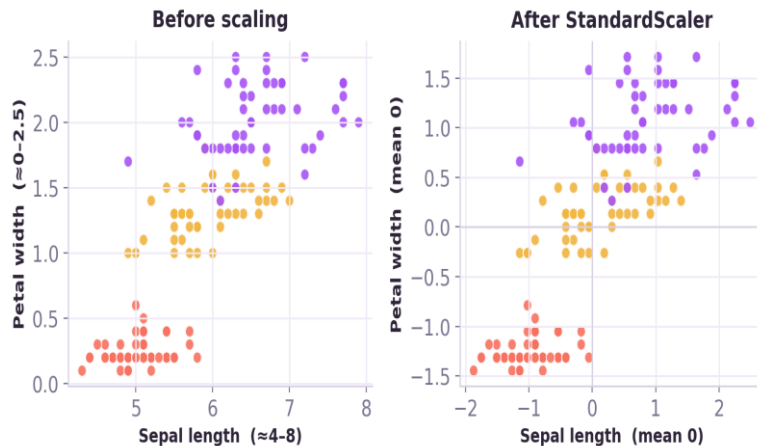


StandardScaler rescales every feature to mean 0, spread 1 — so each one counts equally.



Fit on train, apply to test. Learn the scale from training data only, then reuse it — no peeking (leakage).

Same data, put on an even footing



The Iris Dataset

150

flowers

3

species

4

measurements

THE FOUR MEASUREMENTS (FEATURES → X)

Sepal length

Sepal width

Petal length

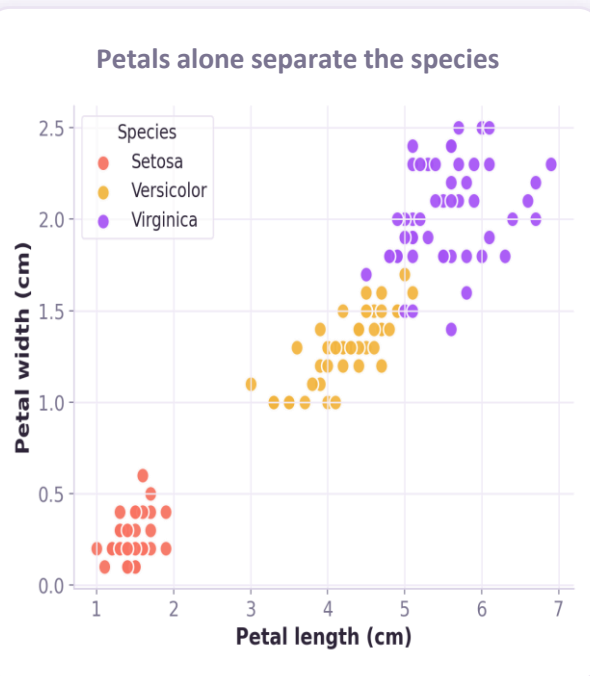
Petal width

THE ANSWER (TARGET → Y)

0 · Setosa

1 · Versicolor

2 · Virginica



THE WORKFLOW

7 steps from raw flowers to a score



STEP 1

Import



STEP 2

Load data



STEP 3

Split



STEP 4

Scale



STEP 5

Train



STEP 6

Predict



STEP 7

Evaluate

Next: every step explained line by line →

K-Nearest Neighbors · Iris Classification



STEP 1 · CODE

Import the tools

imports.py



```
# the libraries we need
from sklearn.datasets import load_iris
from sklearn.model_selection import (
    train_test_split)
from sklearn.preprocessing import (
    StandardScaler)
from sklearn.neighbors import (
    KNeighborsClassifier)
from sklearn.metrics import accuracy_score
```

LINE BY LINE

- **load_iris**
the ready-made flower dataset.
- **train_test_split**
splits data into train vs. test.
- **StandardScaler**
puts every feature on one scale.
- **KNeighborsClassifier**
the KNN algorithm itself.
- **accuracy_score**
measures the % it gets right.



STEP 2 · CODE

Load the dataset

load.py



```
# load the Iris flower dataset
iris = load_iris()

# X = inputs, Y = answers (0/1/2)
X = iris.data
Y = iris.target
```

LINE BY LINE

- **load_iris()**
loads the built-in flower dataset.
- **X = iris.data**
the inputs — the 4 measurements.
- **Y = iris.target**
the answers — species as 0 / 1 / 2.



STEP 3 · CODE

Split: train vs. test

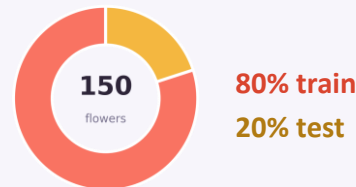
split.py



```
# train on 80%, test on 20%
X_train, X_test, Y_train, Y_test = (
    train_test_split(
        X, Y,
        test_size=0.2,
        random_state=42))
```

LINE BY LINE

- **test_size=0.2**
hold back 20% to test the model.
- **random_state=42**
fixes the split so runs match.
- **4 outputs**
train + test inputs and answers.





STEP 4 · CODE

Scale the features

scale.py



```
# put features on the same scale
scalar = StandardScaler()

# learn scale from train, apply it
X_train = scalar.fit_transform(X_train)

# apply the SAME scale to test
X_test = scalar.transform(X_test)
```

LINE BY LINE

- **StandardScaler()**
creates the scaler object.
- **.fit_transform(X_train)**
learns the scale from training data and rescales it.
- **.transform(X_test)**
applies the SAME scale to test — never re-learned (no leakage).



STEP 5 · CODE

Create & train the model

train.py



```
# k = 5 nearest neighbours vote
knn = KNeighborsClassifier(
    n_neighbors=5)

# fit = memorise the training data
knn.fit(X_train, Y_train)
```

LINE BY LINE

- **n_neighbors=5**
each guess is based on the 5 closest points.
- **.fit(X_train, Y_train)**
“training” here just means storing the scaled training data to compare against later.



STEP 6 · CODE

Make predictions

predict.py



```
# guess labels for unseen flowers  
y_pred = knn.predict(X_test)
```

LINE BY LINE

- **.predict(X_test)**
for each unseen flower, finds its 5 nearest and takes the vote.
- **y_pred**
holds the predicted species (0 / 1 / 2) for every test flower.



STEP 7 · CODE

Check how well it did

evaluate.py



```
# compare guesses vs. real answers
accuracy = accuracy_score(
    y_pred, Y_test) * 100

print("Accuracy of the model is :",
      accuracy, "%")
```

LINE BY LINE

- **accuracy_score(...)**
fraction of test guesses that matched the real answers.
- *** 100**
turns that fraction into a percentage.
- **print(...)**
shows the final score on screen.

100%

accuracy on the 30 test
flowers
(*k* = 5, *random_state*=42)

RECAP

What we built



Neighbours decide

A new flower takes the majority label of its 5 nearest known points.



Scaling is essential

StandardScaler put every feature on equal footing before measuring distance.



Measured the result

Compared the model's guesses to the real labels to score its accuracy.

100%

test accuracy

KNN ($k = 5$) on Iris

